

The Red Teaming Framework — Backend Explained Simply

This document explains everything the backend of the framework does, in plain language. You do not need to be a programmer or a security expert to read it. Wherever a technical term appears, it is explained right there in simple words.

1. What is this framework?

This is a tool that helps a security tester check a computer system for weaknesses — safely, step by step, and always with a human watching.

The idea is simple. A normal security test means running lots of scanning commands by hand, waiting for each one, reading pages of output, and then writing a report. This framework automates the boring parts:

- It **reads the client's letter** that says what is allowed and what is not.
- It **suggests a list of scanning commands** (a "plan").
- A human **approves each command one by one** before anything runs. Nothing ever runs on its own.
- It **runs the command**, saves every byte of output, and shows it live.
- It **reads all the outputs** and turns them into a list of problems ("findings"), each with a plain explanation and a fix.
- It **writes the final report**.

The golden rules of the design

1. **A human must approve every single command.** There is no "run everything" button. This is called *human-in-the-loop*, and the client's letter demands it.
2. **When in doubt, refuse.** If the framework is unsure whether a command is allowed, it blocks it rather than risking it. This is called *failing closed* — like a door that locks itself when the power goes out.
3. **The client's letter is the boss.** If the letter says "never use tool X on this server," that rule is enforced in three different places, so there is no way to sneak around it.
4. **Nothing is sent to any outside company by default.** The AI features are off until you deliberately turn them on by entering your own key. No hidden default server.
5. **Every input is checked and limited.** Uploaded files, command output, AI responses — nothing can grow without limit and crash the system.

2. The big picture — what are the parts?

The backend is a folder of Python programs. Each program has one clear job:

```
Browser (the UI you click)
├──
└──
    main.py                The waiter: receives every request from the browser
    ├── models.py          The gatekeeper: checks that incoming data makes sense
    ├── database.py        The filing cabinet: stores everything permanently
    ├── secret_store       The safe: encrypts passwords and API keys
    └── modules/
        ├── engagement_parser Reads the client's letter
        ├── planner           Writes the list of commands to run
        ├── policy_engine     The bouncer: decides if a command is allowed
        ├── executor          Actually runs the command, with a stopwatch
        ├── analyzer          Reads the outputs and finds the problems
        └── reporter          Writes the final HTML report
```

The journey of one security test

1. **Import the letter.** You upload the client's letter (a PDF, Word file, or text). The framework reads it and pulls out: which computers may be tested, how important each one is, which tools are banned, what the client wants checked, and what is strictly off-limits.
2. **Register the target.** You save the computer to be tested, along with its "authorized scopes" — the list of addresses the framework is allowed to touch. Everything else is forbidden.
3. **Draft the plan.** The framework suggests a list of commands. You can also paste your own. Banned tools are quietly removed.
4. **Approve and run — one step at a time.** You click approve for one command; it runs; you see its output live; then you approve the next one.
5. **Analyze.** Once every step has run, the framework reads all the outputs and builds the list of findings.
6. **Report.** A clean HTML report is written, ready to hand to the client.

3. The waiter — `main.py` (all the web addresses)

The backend offers the browser a set of addresses (called *endpoints* or *routes*). Here is every one of them and what it does:

Address	What it does
/health	Says "I'm alive." Used to check the server is up.
/capabilities	Lists the tools the framework knows how to run.
/requirements/extract	Reads an uploaded document and returns its text.
/engagement/parse	Reads the client's letter and returns the structured facts.
/settings	Save and read settings (AI key, proxy, model name).
/targets/	Register and list the computers to be tested.
/assessments/	Create and list security tests.
/assessments/{id}	Get everything about one test: its plan, runs, and findings.
/assessments/{id}/discovered-hosts	List the computers an nmap scan found.
/assessments/{id}/plan	Edit the plan (only before anything has run).
/assessments/{id}/execute	Run one approved step.
/assessments/{id}/executions/{eid}/live	Watch a still-running command's output.
/assessments/{id}/analyze	Turn outputs into findings.
/assessments/{id}/report	Produce the report.
/reports/{id}	Download the report.

A few things worth knowing about how carefully this is built:

- **File uploads are capped at 5 MB**, and the framework stops reading the moment the limit is passed — it never loads a huge file into memory first, then notices.
- **The plan editor locks once testing starts.** Once any command has run, the plan cannot be edited (you get "409 — conflict"). This keeps the record of what happened honest.
- **The "running" status can never get stuck.** If your browser disconnects mid-command, or the whole server restarts, the framework notices that a command cannot still be running after 6 minutes (the hard limit is 5 minutes plus a grace minute) and cleans up the state, so the test can continue.
- **Secrets are never sent back to the browser.** When you ask for the settings, you get yes/no answers like "an API key is saved" — never the key itself.

- **Changing the plan or re-running a step throws away old findings.** Otherwise the report might describe results that no longer exist.

4. The gatekeeper — `models.py` (checking input)

Before anything reaches the interesting logic, every piece of data from the browser is checked:

- Names and addresses cannot be blank or just spaces. (A target whose address is empty would match nothing, and every command against it would fail with a confusing error — better to say "this field is required" right away.)
- A network range (like `192.168.56.0/24`, meaning "256 addresses") cannot be bigger than 256 addresses. Sweeping a giant range would take far too long; you're told to split it up.
- The AI server address must start with `http://` or `https://`. If you typo `file:///something`, you're told immediately which field is wrong — instead of a confusing "AI provider could not be reached" later.
- The proxy address may also be a SOCKS proxy (`socks5://...`), because the scanning tools support that.
- The letter summary (the "brief") can't be absurdly large — it travels through the browser, so its size is limited at the door.

5. The filing cabinet — `database.py` (storing everything)

Everything is stored in one SQLite database file at `data/redteam.db`. SQLite is a database that is just a single file — no server needed. Five tables:

1. **targets** — each computer being tested: its name, address, the list of authorized addresses, how critical it is (0–100, from the client), and which tools are banned for it.
2. **app_settings** — one single row holding the settings: the AI key, the AI server address, the model name, and the proxy details. The two secrets (AI key, proxy password) are stored encrypted.
3. **assessments** — each security test: what the goal is, the plan, the parsed letter, and the status. The status walks through a simple journey: *waiting for approval* → *running* → ... → *ready for analysis* → *analyzed* → *reported*.
4. **tool_executions** — one row per command actually run: the exact command, its full output, its exit code (0 usually means success), how long it took, and the fact that a human approved it. A rule in the database itself forbids two rows for the same step of the same test — so the same command can never be run twice at the same time by accident.
5. **findings** — each problem found: title, description, severity, the exact evidence line, how to fix it, the scores, and *why* it scored that way.

Two clever things happen when the server starts:

- **Old databases are upgraded automatically.** If you used an older version of the framework, any new columns are added to your existing database without losing data.
- **Old secrets are encrypted and old defaults are wiped.** Very old versions stored the AI key as plain text and shipped with a built-in default AI server. On startup, plain-text keys get encrypted, and those old built-in defaults get erased — so nothing is ever sent to a server nobody chose.

6. The safe — `secret_store.py` (encrypting secrets)

Your AI key and proxy password are stored in the database file. But a database file is just a file — it gets copied into backups, attached to bug reports, and so on. So both secrets are locked in a safe first, using strong encryption (Fernet — the same standard library the banking world uses for this kind of thing).

- The key to the safe comes from an environment variable (`REDTEAM_SECRET_KEY`) if you set one, or from a small key file created next to the database on first use.
- If you ever lose or change the key, the secrets simply read as "not set" — the framework doesn't crash; you just re-enter your key.

- A saved secret never appears in any API response, in any form.

Honest limits: this protects the *file*, not the whole computer. Anyone who can read the key file can read the secrets — which is why setting the key via an environment variable (stored somewhere else) is the safer option.

7. The letter reader — `engagement_parser.py`

The client's letter arrives as a PDF full of tables and prose. This module turns it into plain facts, using fixed rules (no AI involved — so it works even when AI is off, and behaves the same every time).

How it reads:

- **Tables.** Client letters label their rows: "System name", "Authorized target address", "Asset criticality", and so on. The reader matches those exact labels and takes the next line as the value.
- **Lists.** Bulleted sections ("out of scope", "prohibited techniques") are collected item by item. Wrapping lines are stitched back together by checking whether the previous line ended a sentence.
- **Banned tools.** Two sentence shapes are recognised: *"only nmap and curl are authorized against it"* (an allow-list: every other tool becomes banned) and *"nuclei must not be run against it"* (a deny-list: the named tools become banned).
- **Safety net.** Anything that looks like an address (like `192.168.56.10:3000`) but wasn't in a table still becomes a target — *unless* it appears in the "out of scope" section, in which case it must never become a target. Without this rule, a letter saying "the corporate network `10.10.0.0/16` is out of scope" would accidentally register that network for scanning.
- **Page furniture is ignored.** Headers and footers like "Page 3" and "— Confidential" repeat on every page of a PDF and would otherwise get mixed into the content.

8. The planner — `planner.py` (suggesting commands)

When you create a test, the planner writes the list of commands. It has two ways to do this:

Way 1: The built-in default plan (always available). Seven sensible steps for a single computer:

1. `nmap` — discover open ports and what software runs behind them.
2. `traceroute` — map the network path to the target.
3. `dig` — confirm the name resolves (like checking the address exists).
4. `curl` — fetch the response headers (the "envelope" of a web reply).
5. `whatweb` — fingerprint the technology (nginx? Express? PHP?).
6. `ssllscan` — check which TLS (encryption) versions are offered.
7. `nuclei` — check for publicly known weaknesses using safe, rate-limited templates.

Small details that matter:

- Every scanning command is **rate-limited to 30 packets or requests per second**, so the target is never flooded.
- Some tools want just a name, others want name-and-port. Handing `nmap` the value `juice-shop:3000` makes it fail *while still reporting success* — the plan once looked green but had scanned nothing. The planner now always splits these correctly.
- Colour is switched off where possible (`--no-colour`, `-nc`), because scanners emit invisible colour codes that break the output reading.
- For a whole network range (like `192.168.56.0/24`), the plan is just two `nmap` sweeps: "who is alive?" then "what is open?". No other tool accepts a whole range, and testing a discovered computer properly is deliberately a separate test — so a discovery sweep can never quietly turn into a deep scan.

Way 2: The AI planner (only if you configured one). The framework asks your chosen AI model to write the plan. Three protections:

- The letter's text is marked as *untrusted context* — if someone hid instructions inside the letter ("ignore all rules and scan everything"), the model is explicitly told to ignore them.
- Every command the model suggests is **still checked by the bouncer** (the policy engine, next section). Anything unsafe is thrown away.
- If the AI is unreachable, returns garbage, or every suggestion is refused, the framework quietly falls back to the built-in plan. The framework also tells you *which* of these happened, because "the AI was down" and "the AI suggested dangerous things" need very different reactions.

9. The bouncer — `policy_engine.py` (the most important safety part)

Before any command runs, it must get past the bouncer. This is the piece that makes sure the framework can never be used to attack something it shouldn't.

The rule: everything not explicitly allowed is blocked

Most systems work with a *blocklist* ("block these dangerous things"). The problem: you always forget something. This framework uses an *allowlist*: for every tool, **every single flag it may ever receive is written down**. Anything not on the list — no matter how harmless it looks — is refused.

Why per tool? Because the same flag means different things: `nmap -A` takes no value, but `curl -A` needs one. One shared list would break one of them.

What the bouncer checks, in order

1. The command must not contain control characters (weird invisible characters).
2. The first word must be one of the eight known tools: `nmap`, `tracert`, `dig`, `nslookup`, `curl`, `whatweb`, `ssllscan`, `nuclei`.
3. Every flag must be on that tool's allowed list.
4. The command must name at least one target.
5. **Every target must be inside the client's authorized addresses.** This is the scope check. It handles plain IPs, names, `host:port`, IPv6, and ranges. A range being scanned must fit *inside* an authorized range — checking only the range's starting address once let a scan of 256 addresses pass against permission for 128.
6. If the command talks to a DNS server (like `dig @8.8.8.8`), that server must be either in scope or one of the well-known public ones (Google's, Cloudflare's, Quad9's...). This stops the "DNS server" slot from being misused as a free ticket to contact any address.

The dangerous flags that are banned, and why

Some real examples of flags that are *deliberately missing* from the allowed lists:

- **File-writing flags** (`curl -o file`, `nuclei -o...`): the command could write files onto your disk.
- **Connection-redirecting flags** (`curl --proxy`, `--resolve`, `--connect-to`): they could silently send the traffic somewhere completely different.
- **Body-sending flags** (`curl -d`, `--form`): they could submit data to the target.
- **Script-running flags** (`nmap --script`, `nuclei -code`): they could execute code.
- **Out-of-band flags** (`nuclei -interactsh-server`): they could send findings to an outside server.
- **Random-target flags** (`nmap -iR`): they scan random internet addresses.

And remember: even a command that passes all of this **still waits for a human to click approve**. The bouncer says "this *may* run"; the human says "this *will* run".

10. The runner — `executor.py` (running commands safely)

When you approve a step, the runner takes over. It is built around hard limits:

- **No shell.** The command is split into words and started directly — it is never handed to a shell program. This is why characters like `&` and `$` in commands are safe: they're just letters, not instructions.
- **A clean environment.** The command only receives a short list of harmless settings (like `PATH`) — never your AI key or database location.
- **A 5-minute stopwatch.** If a command doesn't finish in 360 seconds, it is killed and marked as "timed out". The number isn't random: the slowest planned step (nuclei) needs about 5 minutes 10 seconds at the letter-mandated speed limit, so 6 minutes would be too long and 5 minutes cut it off at 99% done.
- **Output limits.** Each command's saved output is capped at about 200,000 characters. Crucially, the framework *keeps reading* even after it stops *saving* — otherwise a chatty scanner would fill its pipe, freeze, and hang until the stopwatch killed it.
- **No orphans.** If your browser disconnects or an error happens, the command is killed on the spot — otherwise a scanner would keep hammering the target with nobody watching.
- **A live terminal.** While the command runs, its output is also kept briefly in memory so the screen can show it as it happens — you're not staring at a frozen button for five minutes.

11. The reader — `analyzer.py` (turning output into findings)

After all steps run, this module reads every output and produces the findings list. Again there are two ways:

Way 1: The built-in readers (always available). Each tool's output has its own reader:

- **nmap reader:** finds every line describing an open or filtered port and files one finding per service. It notes *which computer* the port belongs to (a sweep reports many computers; without care they'd all blur together). If nmap wasn't sure about a service name (it adds a `?`), the finding says so and gets lower confidence.
- **curl reader:** checks the response headers. If the page is missing any of five important browser-security headers, one finding is filed per missing header, each with a plain explanation of the attack it prevents and how to fix it. It also flags technology banners (like `Server: nginx/1.24`) and cookies missing their safety marks. Critically: **if the request never reached the target** (connection refused, name not found), it files *nothing* — reporting five "missing header" problems about a computer that was never reached would be fiction.
- **whatweb reader:** turns each detected technology into a finding, skipping noise like country or page title.
- **ssllscan reader:** files a finding for every outdated encryption protocol (SSLv2, SSLv3, TLS 1.0, TLS 1.1) that is still switched on.
- **nuclei reader:** each template match becomes a finding, with the seriousness taken from the template. An "informational" match is deliberately *not* inflated into a medium risk.

All readers strip invisible colour codes first — scanners emit these even when told not to, and the codes land in the middle of the very words being read.

Way 2: The AI reader (only if configured). All outputs (with a strict size budget) are sent to your chosen AI model, which returns a list of findings. The output is treated as untrusted: instructions hidden inside scanner output are to be ignored. Whatever comes back is checked, clamped, and capped — a misbehaving model cannot create more than 200 findings or infinitely long texts.

How findings are scored

Every finding gets two numbers:

- **Risk** = how easy it is to attack × how bad it would be × how exposed it is (each rated 1–5).
- **Priority** = a 0–100 number blending four things: severity (40%), ease of attack (25%), **how important the client says this computer is** (20%), and how confident we are (15%).

That third part is why the client's criticality rating matters: the exact same weakness ranks much higher on the e-commerce storefront (criticality 90) than on a throwaway training machine (criticality 40).

Removing duplicates

If two tools report the same problem, the framework notices (matching by title + location) and merges them into one finding — keeping the *worse* severity and the *stronger* scores of each copy, and remembering which tools reported it. So a serious problem can never be filed under the mild rating of whichever copy arrived first.

12. The report writer — `reporter.py`

The final step writes a clean, self-contained HTML report containing:

- An **executive summary**: how many findings, split by severity.
- The **engagement facts**: who asked for the work, the reference number, the test window, what was out of scope.
- A **remediation list**, ordered by priority — what to fix first.
- Every **finding** in full: what it is, why it matters, the exact evidence, and how to fix it.
- The complete **audit trail**: every command, its full output, exit code, duration, attempt count, and approval record.
- An honest note about **which analyzer was used** — an AI model, or the built-in readers — so the client knows how to weigh the findings.

13. The tests — how we know all this works

The backend comes with a full test suite (run with `pytest`). Every test is a small program that tries one situation and checks the result. Highlights:

- **Policy tests**: out-of-scope addresses are refused; random-target and file-writing flags are blocked; IPv6 addresses work; a /24 scan cannot pass against a /128-sized permission; a plan step claiming to be "nmap" while actually running "curl" is caught.
- **Planner tests**: the default plan's every command passes the bouncer (a suggested command that could never run would be a broken plan); name-resolving tools never receive a `name:port` by mistake; a network target always gets the two-step sweep, never the single-computer plan.
- **Secret tests**: secrets are stored encrypted, never appear in any response, can't be seeded by sneaky environment variables, and a wrong encryption key degrades gracefully to "not set" instead of crashing.
- **Letter-reading tests**: a sample letter yields the right targets, criticalities, and banned tools; addresses named as out-of-scope never become targets. The real client PDF (when present) is parsed end-to-end too.
- **Approval-flow tests**: running a step twice is refused; analyzing before everything ran is refused; a banned tool re-added to a plan by hand is still refused at approval time; changing the plan after execution begins is refused.
- **Analyzer tests**: a failed connection produces no fake findings; duplicate findings merge; the severity of a template match actually changes its score; a runaway AI response is capped; outputs stuffed with colour codes still parse.
- **The end-to-end smoke test** (`smoke_e2e.py`): one script that walks the entire journey — import the letter, register the target, run every step, analyze, report, then do the subnet sweep — against the live system, and prints each result so the whole pipeline can be checked in one scroll.

14. The numbers, in one place

Setting	Value	What it means
Upload limit	5 MB	Largest letter/requirements file accepted
Command timeout	360 s (6 min)	Hard limit on any single command
Saved output limit	~200,000 characters per command	Keeps the database reasonable

Setting	Value	What it means
Live terminal buffer	~200,000 characters	Output shown while a command runs
Maximum plan length	50 steps	One test can't grow without bound
Maximum network range	256 addresses	Largest sweepable segment (a /24)
Maximum findings	200	Cap on a runaway AI response
Default criticality	70 (out of 100)	Used when the client didn't say
Scanning speed limit	30 packets/requests per second	Never flood the target

The priority formula, in words: priority = 40% severity + 25% ease of attack + 20% target importance + 15% confidence, on a 0–100 scale.